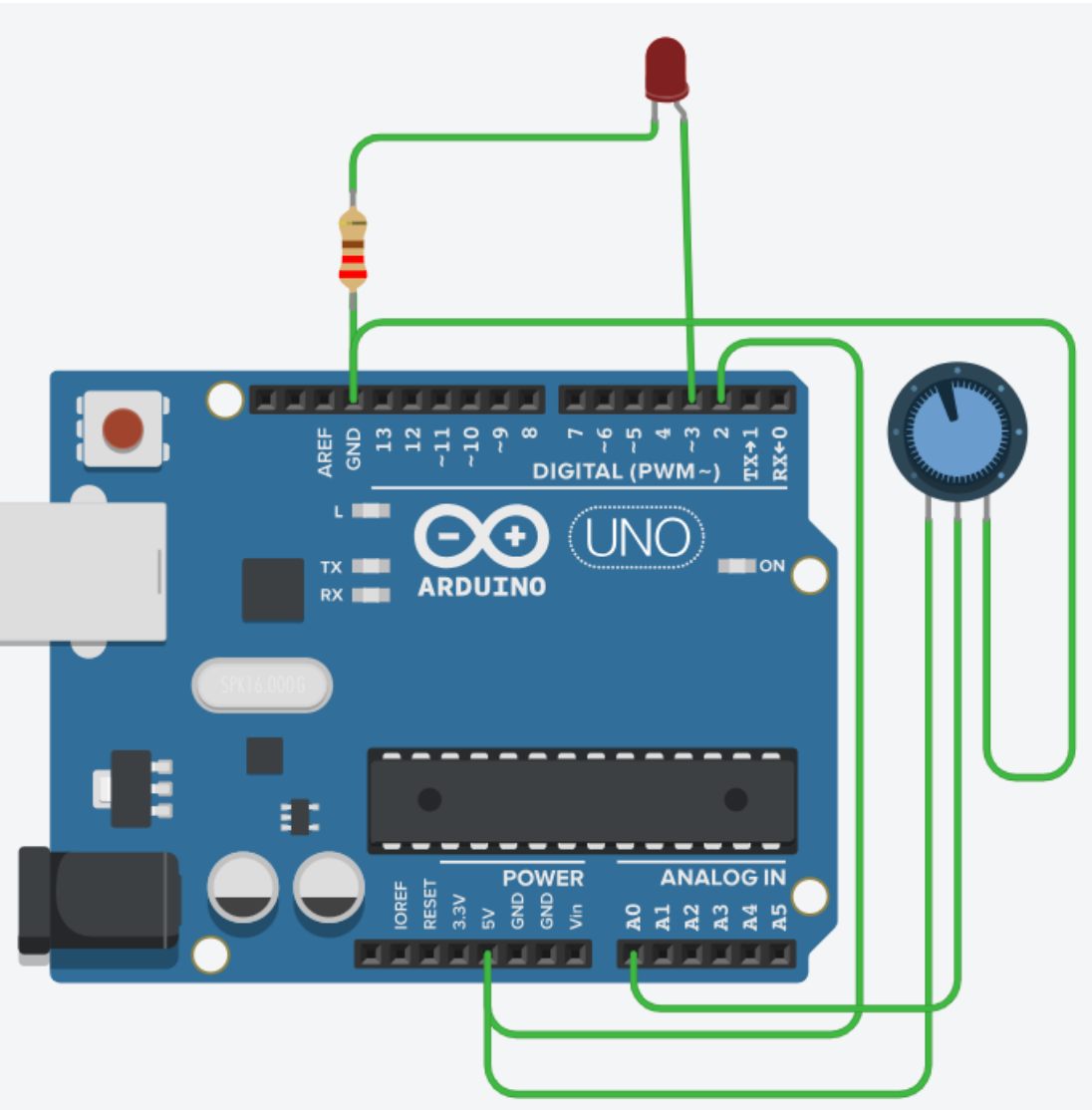


Exercise 1: Adjustable LED Blinker Using Arduino

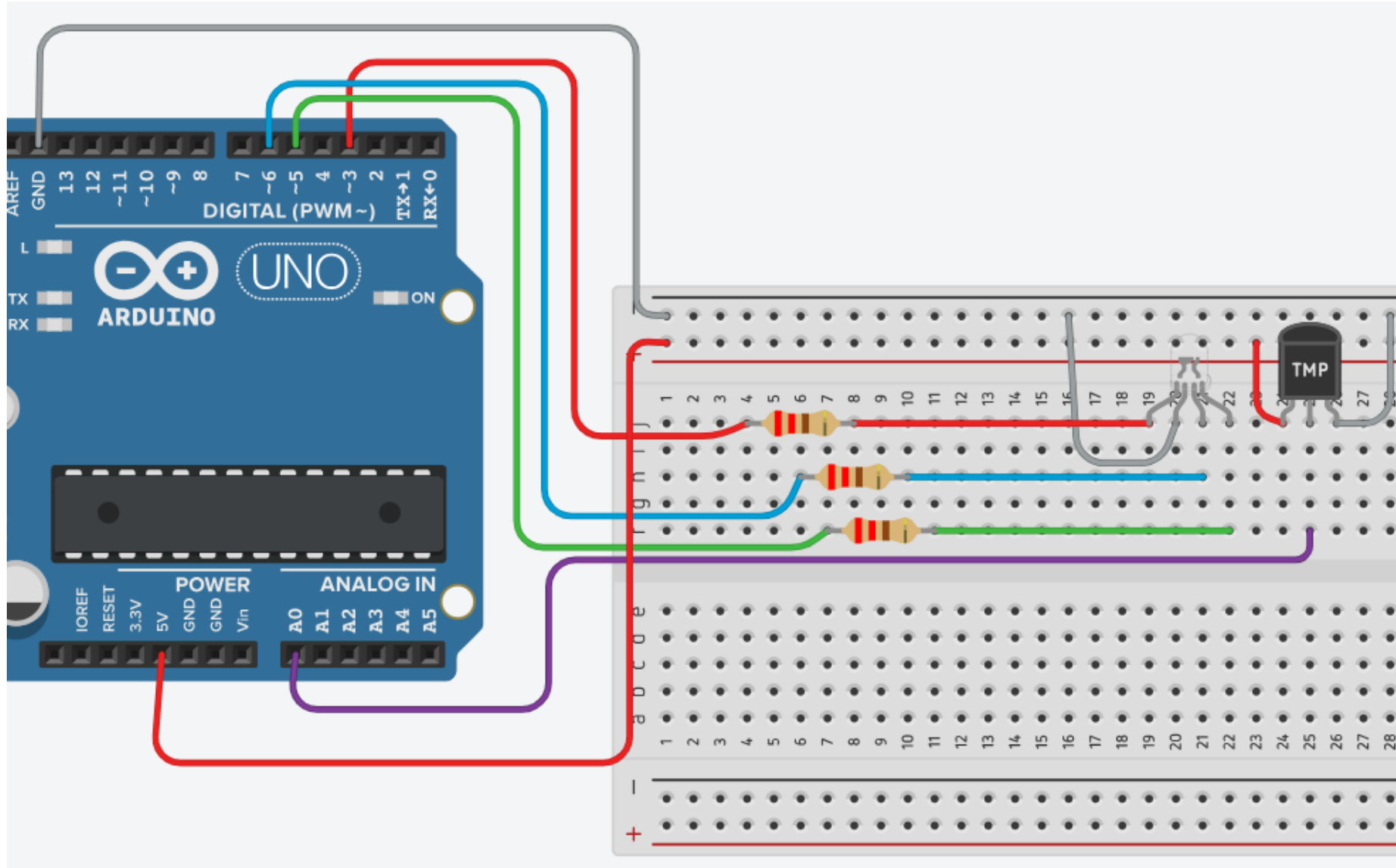
Design and program an Arduino-based system where the blinking frequency of an LED is dynamically controlled by a potentiometer.



```
// Global variables
int potReading = 0 ;
void setup()
{
  pinMode(3, OUTPUT); // Initialize digital pin 3 as an output for the LED
  Serial.begin(9600);
}
void loop()
{
  // Read the position of the potentiometer (Value between 0 and 1023)
  potReading = analogRead(A0);
  // Print the reading to the Serial Monitor for debugging
  Serial.print("Potentiometer Reading: ");
  Serial.println(potReading);
  digitalWrite(3, HIGH); // Turn the LED on
  // Wait based on the potentiometer reading
  delay(potReading);
  digitalWrite(3, LOW); // Turn the LED off
  delay(potReading); // Wait again based on the potentiometer reading
}
```

Exercise 2: Temperature-Controlled RGB/Multi-LED Indicator Using Arduino

Design and program an Arduino system that reads data from an analog temperature sensor (like a TMP36), converts the voltage into Celsius, and lights up three different LEDs (or an RGB LED) depending on the temperature range.



Exercise 2: Temperature-Controlled RGB/Multi-LED Indicator Using Arduino

Design and program an Arduino system that reads data from an analog temperature sensor (like a TMP36), converts the voltage into Celsius, and lights up three different LEDs (or an RGB LED) depending on the temperature range.

```
// Define clear pin names for the RGB LED & Global variables for sensor math
```

```
int RED_PIN  = 6;  
int GREEN_PIN = 5;  
int BLUE_PIN = 3;  
double rawReading;  
double vout;  
double tempCelsius;
```

```
void setup()  
{  
  pinMode(A0, INPUT);
```

```
// Set RGB pins as outputs
```

```
pinMode(RED_PIN, OUTPUT);  
pinMode(GREEN_PIN, OUTPUT);  
pinMode(BLUE_PIN, OUTPUT);
```

```
Serial.begin(9600);
```

```
}
```

Exercise 2: Temperature-Controlled RGB/Multi-LED Indicator Using Arduino

Design and program an Arduino system that reads data from an analog temperature sensor (like a TMP36), converts the voltage into Celsius, and lights up three different LEDs (or an RGB LED) depending on the temperature range.

```
void loop()
{
  // 1. Read the temperature sensor (TMP36)
  rawReading = analogRead(A0);
  vout = rawReading * 5.0 / 1023.0;
  tempCelsius = (vout - 0.5) * 100.0;

  // Print temperature to Serial Monitor
  Serial.print("Temperature: ");
  Serial.print(tempCelsius);
  Serial.println(" C");

  if (tempCelsius < 11.0) // Condition 1: COLD (< 11°C) -> Turn on BLUE
  {
    digitalWrite(BLUE_PIN, HIGH);
    digitalWrite(GREEN_PIN, LOW);
    digitalWrite(RED_PIN, LOW);
  }
```

Exercise 2: Temperature-Controlled RGB/Multi-LED Indicator Using Arduino

Design and program an Arduino system that reads data from an analog temperature sensor (like a TMP36), converts the voltage into Celsius, and lights up three different LEDs (or an RGB LED) depending on the temperature range.

```
// Condition 2: COMFORT (11°C to 20°C) -> Turn on GREEN
```

```
else if (tempCelsius >= 11.0 && tempCelsius <= 20.0)
```

```
{
```

```
    digitalWrite(BLUE_PIN, LOW);
```

```
    digitalWrite(GREEN_PIN, HIGH);
```

```
    digitalWrite(RED_PIN, LOW);
```

```
}
```

```
// Condition 3: HOT (> 20°C) -> Turn on RED
```

```
else if (tempCelsius > 20.0)
```

```
{
```

```
    digitalWrite(BLUE_PIN, LOW);
```

```
    digitalWrite(GREEN_PIN, LOW);
```

```
    digitalWrite(RED_PIN, HIGH);
```

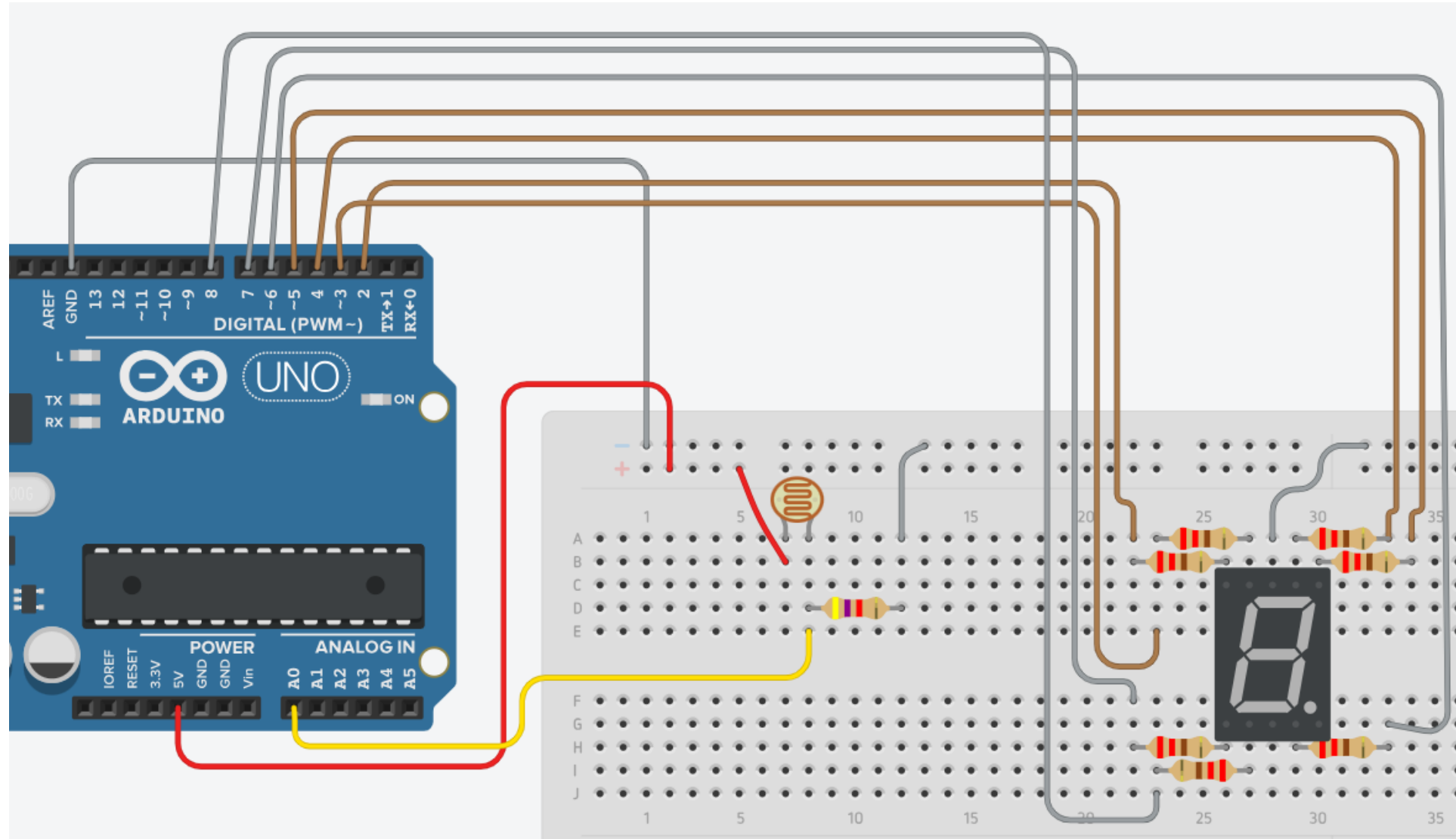
```
}
```

```
delay(500); // Check temperature every half second
```

```
}
```

Exercise 3: Light Level Indicator Using a 7-Segment Display

Design and program an Arduino system that measures ambient light levels using a Light Dependent Resistor (LDR) and displays the corresponding brightness level (from 0 to 9) on a 1-digit 7-segment display.



Exercise 3: Light Level Indicator Using a 7-Segment Display

Design and program an Arduino system that measures ambient light levels using a Light Dependent Resistor (LDR) and displays the corresponding brightness level (from 0 to 9) on a 1-digit 7-segment display.

```
int ldrPin = A0;    // Define the analog pin for the Light Dependent Resistor (LDR)
int ldrReading = 0; // Variable to store the raw analog value (0 to 1023)
int level = 0;      // Variable to store the mapped brightness level
```

```
// Define digital pin assignments for each segment of the 7-segment display
```

```
int A = 4;
int B = 5;
int C = 6;
int D = 8;
int E = 7;
int F = 3;
int G = 2;
```

```
void setup()
{
  pinMode(ldrPin, INPUT); // Configure the LDR pin as an input
  Serial.begin(9600);
  // Configure all the 7-segment display pins as outputs
  pinMode(A, OUTPUT);
  pinMode(B, OUTPUT);
  pinMode(C, OUTPUT);
  pinMode(D, OUTPUT);
  pinMode(E, OUTPUT);
  pinMode(F, OUTPUT);
  pinMode(G, OUTPUT);
}
```

Exercise 3: Light Level Indicator Using a 7-Segment Display

Design and program an Arduino system that measures ambient light levels using a Light Dependent Resistor (LDR) and displays the corresponding brightness level (from 0 to 9) on a 1-digit 7-segment display.

```
void loop()
{
  // 1. Read the raw light value from the LDR sensor
  ldrReading = analogRead(ldrPin);

  // 2. Map the sensor reading (0-1023) to a single-digit range (0-9)
  level = map(ldrReading, 0, 1023, 0, 9);

  // 3. Check the mapped level and call the matching display function
  if (level == 0) display0();
  if (level == 1) display1();
  if (level == 2) display2();
  if (level == 3) display3();
  if (level == 4) display4();
  if (level == 5) display5();
  if (level == 6) display6();
  if (level == 7) display7();
  if (level == 8) display8();
  if (level == 9) display9();

  Serial.print("Current Light Level: ");
  Serial.println(level);
  delay(100); // Wait for 100 milliseconds before taking the next reading
}
```

Exercise 3: Light Level Indicator Using a 7-Segment Display

Design and program an Arduino system that measures ambient light levels using a Light Dependent Resistor (LDR) and displays the corresponding brightness level (from 0 to 9) on a 1-digit 7-segment display.

// --- Functions to control individual segments for each number ---
// (1 turns a segment HIGH/ON, 0 turns it LOW/OFF)

```
void display0(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, HIGH);  
  digitalWrite(F, HIGH);  
  digitalWrite(G, LOW);  
}
```

```
void display2(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, LOW);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, HIGH);  
  digitalWrite(F, LOW);  
  digitalWrite(G, HIGH);  
}
```

```
void display4(){  
  digitalWrite(A, LOW);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, LOW);  
  digitalWrite(E, LOW);  
  digitalWrite(F, HIGH);  
  digitalWrite(G, HIGH);  
}
```

```
void display6(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, LOW);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, HIGH);  
  digitalWrite(F, HIGH);  
  digitalWrite(G, HIGH);  
}
```

```
void display8(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, HIGH);  
  digitalWrite(F, HIGH);  
  digitalWrite(G, HIGH);  
}
```

```
void display1(){  
  digitalWrite(A, LOW);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, LOW);  
  digitalWrite(E, LOW);  
  digitalWrite(F, LOW);  
  digitalWrite(G, LOW);  
}
```

```
void display3(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, LOW);  
  digitalWrite(F, LOW);  
  digitalWrite(G, HIGH);  
}
```

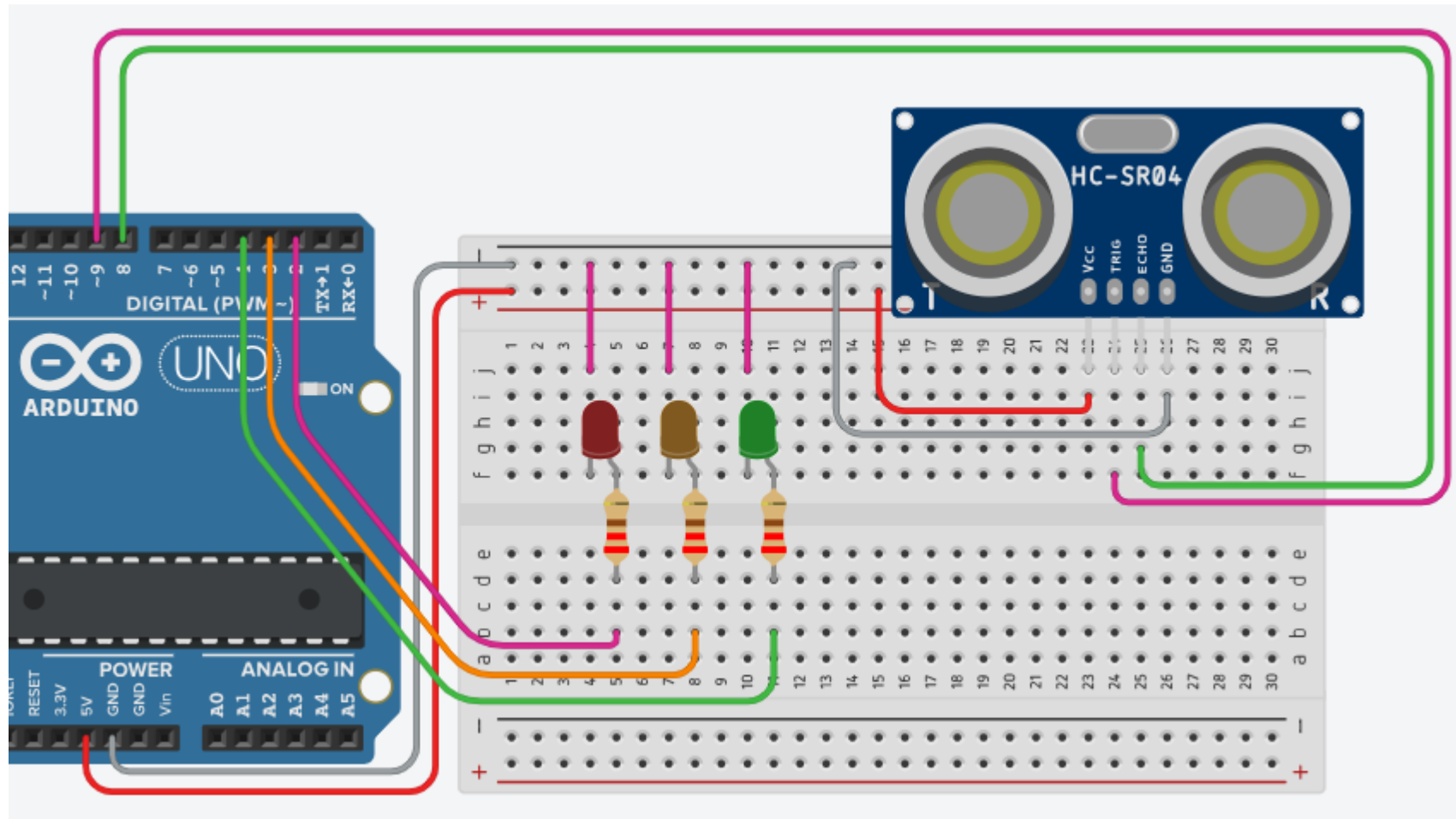
```
void display5(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, LOW);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, LOW);  
  digitalWrite(F, HIGH);  
  digitalWrite(G, HIGH);  
}
```

```
void display7(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, LOW);  
  digitalWrite(E, LOW);  
  digitalWrite(F, LOW);  
  digitalWrite(G, LOW);  
}
```

```
void display9(){  
  digitalWrite(A, HIGH);  
  digitalWrite(B, HIGH);  
  digitalWrite(C, HIGH);  
  digitalWrite(D, HIGH);  
  digitalWrite(E, LOW);  
  digitalWrite(F, HIGH);  
  digitalWrite(G, HIGH);  
}
```

Exercise 4: Distance-Based Indicator Using an Ultrasonic Sensor

Design and program an Arduino system that measures target distance using an HC-SR04 ultrasonic sensor and triggers different LED behaviors (including flashing for close proximity) depending on the calculated distance.



Exercise 4: Distance-Based Indicator Using an Ultrasonic Sensor

Design and program an Arduino system that measures target distance using an HC-SR04 ultrasonic sensor and triggers different LED behaviors (including flashing for close proximity) depending on the calculated distance.

```
int trigPin = 9; // HC-SR04 trigger pin
int echoPin = 8; // HC-SR04 echo pin
float duration, distance;

void setup()
{
  // Configure the ultrasonic sensor control pins
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an OUTPUT
  pinMode(echoPin, INPUT); // Sets the echoPin as an INPUT

  // Configure digital pins 2, 3, and 4 as outputs for the indicator LEDs
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);

  // Initialize serial communication at 9600 bits per second
  Serial.begin(9600);
}
```

Exercise 4: Distance-Based Indicator Using an Ultrasonic Sensor

Design and program an Arduino system that measures target distance using an HC-SR04 ultrasonic sensor and triggers different LED behaviors (including flashing for close proximity) depending on the calculated distance.

```
void loop()
{
  // --- Trigger the Ultrasonic Sensor ---
  digitalWrite(trigPin, LOW); // Clears the trigPin condition to ensure a clean high pulse
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH); // Set the trigger pin HIGH for 10µs to send the sonic burst
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // --- Read the Echo and Calculate Distance ---
  duration = pulseIn(echoPin, HIGH); // Measure the time (in microseconds) the echo pin stays HIGH
  distance = (duration / 2.0) * 0.0343; // Convert echo travel time to distance in centimeters (speed of sound = 0.0343 cm/µs)

  // Print the calculated distance to the Serial Monitor
  Serial.println(distance);
}
```

Exercise 4: Distance-Based Indicator Using an Ultrasonic Sensor

Design and program an Arduino system that measures target distance using an HC-SR04 ultrasonic sensor and triggers different LED behaviors (including flashing for close proximity) depending on the calculated distance.

```
// --- LED Indicator Logic Based on Distance Ranges ---  
// Condition 1: Out of range (Beyond maximum reliable sensor limit or too close)  
if(distance >= 330 || distance < 2)  
{  
  Serial.println("Out of range");  
  digitalWrite(2, LOW); // Turn off all indicator LEDs  
  digitalWrite(3, LOW);  
  digitalWrite(4, LOW);  
}  
else if(distance < 50) // Condition 2: Close Proximity (Less than 50 cm) -> Flash the LED on pin 2  
{  
  digitalWrite(2, HIGH); // LED 2 ON  
  digitalWrite(3, LOW);  
  digitalWrite(4, LOW);  
  delay(80); // Quick flash delay  
  digitalWrite(2, LOW); // LED 2 OFF  
  digitalWrite(3, LOW);  
  digitalWrite(4, LOW);  
  delay(80); // Quick flash delay  
  digitalWrite(2, HIGH); // LED 2 back ON  
  digitalWrite(3, LOW);  
  digitalWrite(4, LOW);  
}
```

Exercise 4: Distance-Based Indicator Using an Ultrasonic Sensor

Design and program an Arduino system that measures target distance using an HC-SR04 ultrasonic sensor and triggers different LED behaviors (including flashing for close proximity) depending on the calculated distance.

```
// Condition 3: Warning Zone (Between 50 cm and 100 cm) -> Turn on LED on pin 3
```

```
else if(distance >= 50 && distance < 100)
```

```
{
```

```
  digitalWrite(2, LOW);
```

```
  digitalWrite(3, HIGH); // LED 3 ON
```

```
  digitalWrite(4, LOW);
```

```
}
```

```
// Condition 4: Safe Zone (Greater than 100 cm) -> Turn on LED on pin 4
```

```
else if(distance > 100)
```

```
{
```

```
  digitalWrite(2, LOW);
```

```
  digitalWrite(3, LOW);
```

```
  digitalWrite(4, HIGH); // LED 4 ON
```

```
}
```

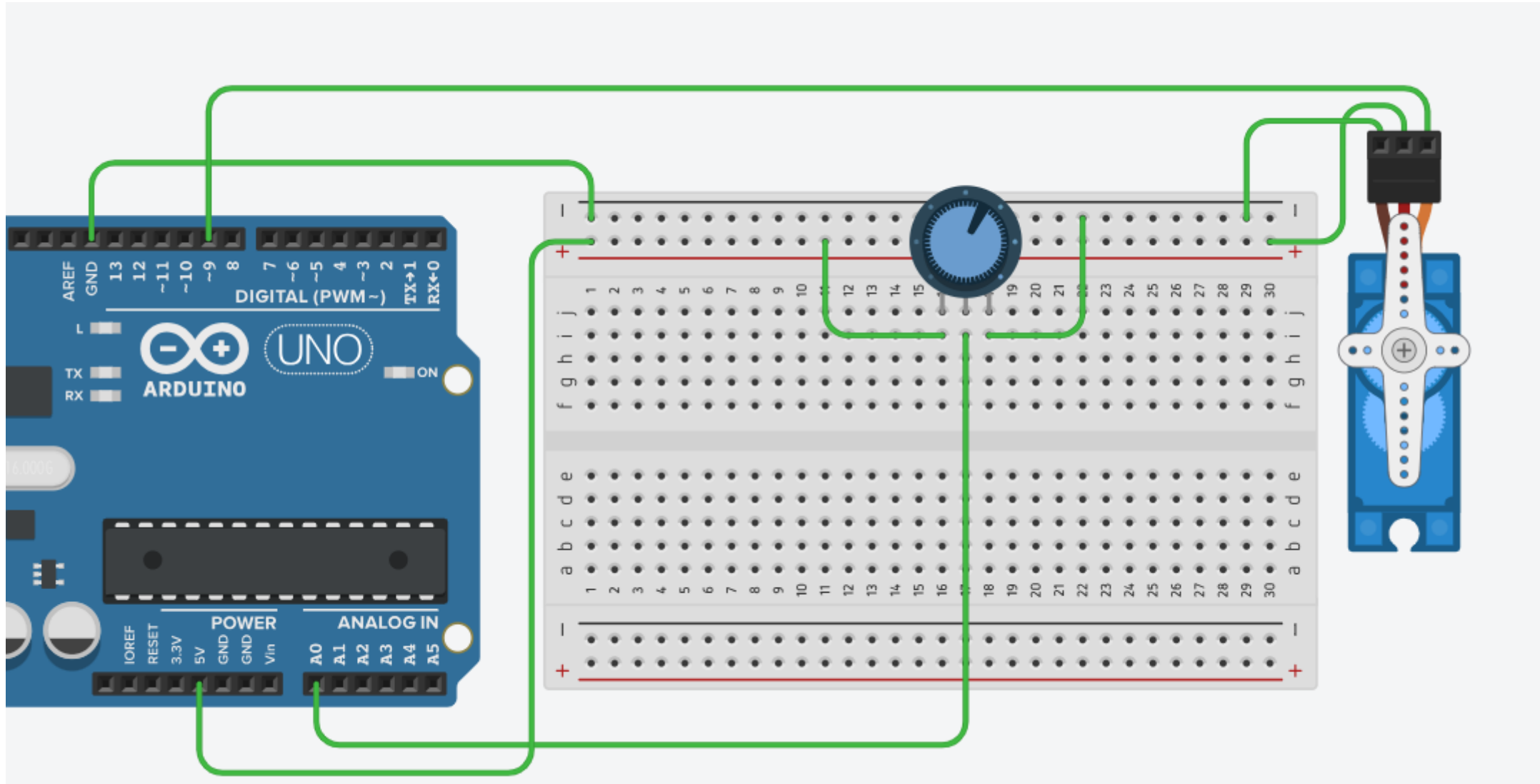
```
// Short delay before running the next loop measurement
```

```
delay(100);
```

```
}
```

Exercise 5: Potentiometer-Controlled Servo Positioner

Design and program an Arduino system where the angular position of a servo motor (0 to 180) is directly and dynamically controlled by turning a potentiometer.



Exercise 5: Servo Motor Sweep Control Using Arduino

Design and program an Arduino system that automatically controls a servo motor to sweep back and forth continuously between 0° and 180°.

```
#include <Servo.h> // Include the standard Arduino Servo library
Servo servo;      // Create a servo object to control the motor

int servoPin = 9; // Servo motor signal pin
int potPin = A0;  // Potentiometer analog input pin
int potReading;   // Variable to store raw data from potentiometer
int angle;        // Variable to store the calculated servo angle

void setup()
{
    servo.attach(servoPin); // Define servo motor pin to Servo library
}

void loop()
{
    potReading = analogRead(potPin); // 1. Read the analog value from the potentiometer (Value between 0 and 1023)
    angle = map(potReading, 0, 1023, 0, 180); // 2. Map the sensor reading to a range for the Servo angle (0 to 180 degrees)
    servo.write(angle); // 3. Command the servo to rotate to the mapped angle position

    // Small delay to allow the servo motor to physically reach the position
    delay(15);
}
```

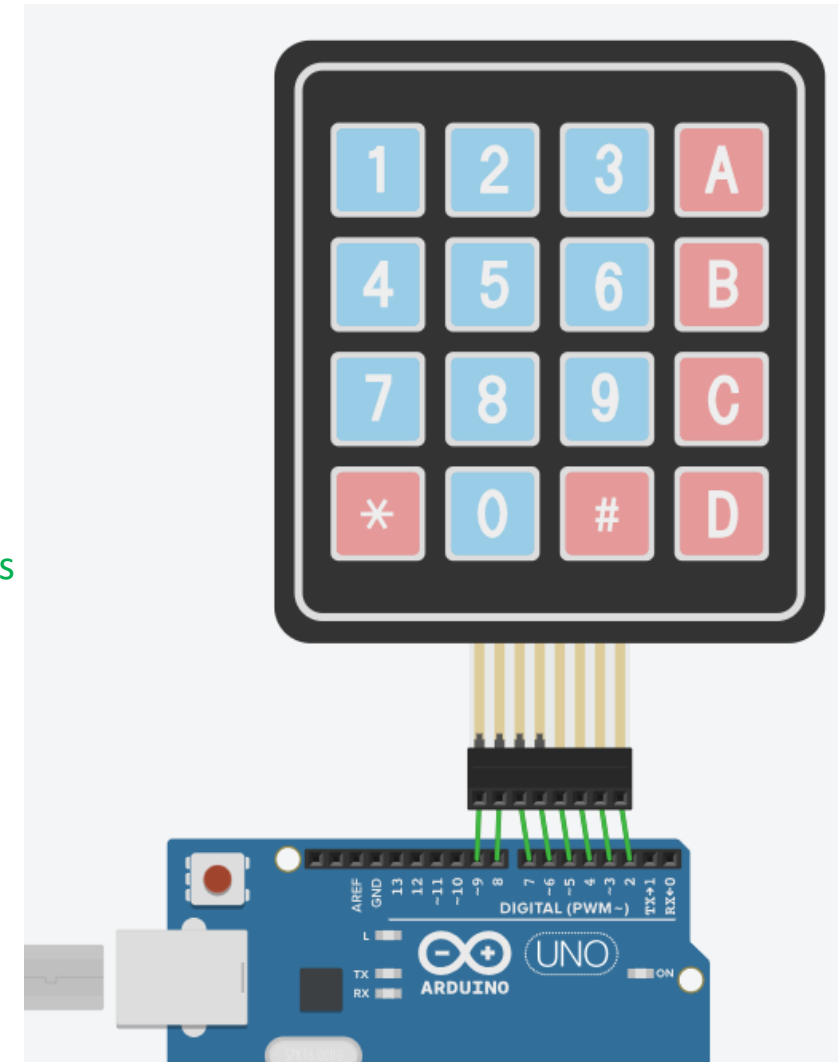
Exercise 6: Matrix Keypad Input Using Arduino

Design and program an Arduino system that interfaces with a 4 x 4 matrix keypad using the Keypad.h library to detect key presses and display the corresponding characters on the Serial Monitor.

```
#include <Keypad.h> // Include the external Keypad library to handle the matrix layout
const byte ROWS = 4; // Define the total number of rows on the keypad
const byte COLS = 4; // Define the total number of columns on the keypad
// Define the precise layout symbols of the keys as they appear on the matrix
char hexaKeys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','D','#','D'}
};
byte rowPins[ROWS] = {9, 8, 7, 6}; // Connect the physical row pins of the keypad to the pins
byte colPins[COLS] = {5, 4, 3, 2}; // Connect the physical column pins of the keypad to the pins

// Initialize an instance of the Keypad class with the custom configurations
Keypad customKeypad = Keypad( makeKeymap(hexaKeys), rowPins, colPins, ROWS, COLS);

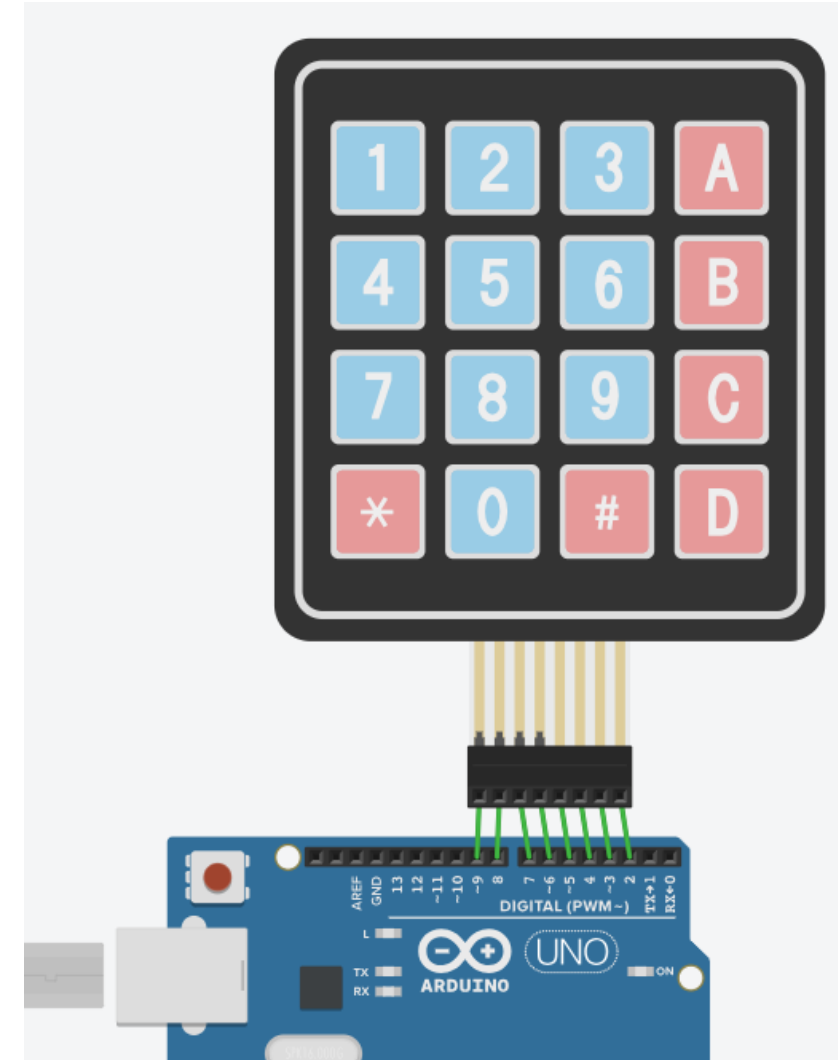
void setup(){
  Serial.begin(9600);
}
```



Exercise 6: Matrix Keypad Input Using Arduino

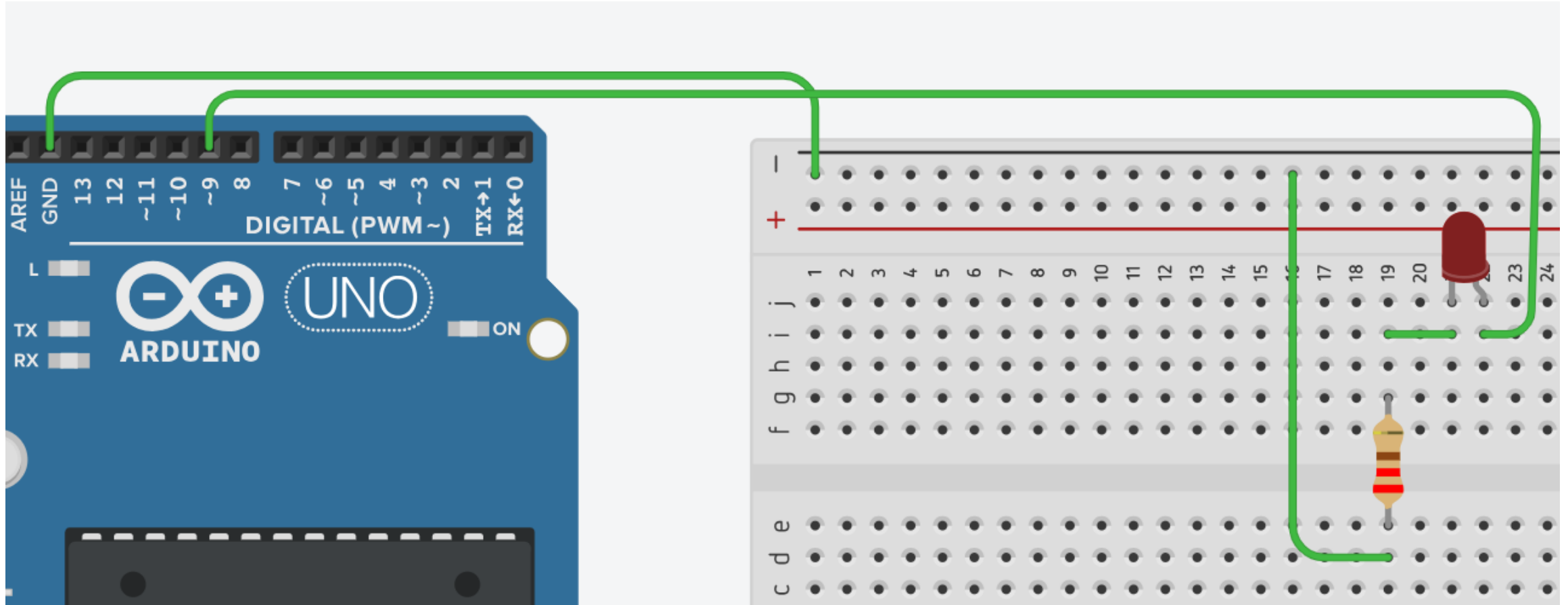
Design and program an Arduino system that interfaces with a 4 x 4 matrix keypad using the Keypad.h library to detect key presses and display the corresponding characters on the Serial Monitor.

```
void loop(){  
  // Check the matrix grid to see if any key has been pressed  
  char customKey = customKeypad.getKey();  
  
  // If a valid key press is captured (value is not null), execute the code block  
  if (customKey){  
    // Output the specific character of the pressed key to the Serial Monitor  
    Serial.println(customKey);  
  }  
}
```



Exercise 7: Stepped LED Brightness Control Using PWM

Design and program an Arduino system that uses Pulse Width Modulation (PWM) to step up the brightness of an LED through five distinct levels before turning it off completely.



Exercise 7: Stepped LED Brightness Control Using PWM

Design and program an Arduino system that uses Pulse Width Modulation (PWM) to step up the brightness of an LED through five distinct levels before turning it off completely.

```
void setup()
{
  // Configure digital pin 9 as an output. Pin 9 supports hardware PWM (Pulse Width Modulation) on the Arduino Uno.
  pinMode(9, OUTPUT);
}

void loop()
{
  // Step 1: Set brightness to ~20% duty cycle (51 / 255)
  analogWrite(9, 51);
  delay(500); // Hold this brightness for 500 milliseconds

  // Step 2: Set brightness to ~40% duty cycle (102 / 255)
  analogWrite(9, 102);
  delay(500); // Hold this brightness for 500 milliseconds

  // Step 3: Set brightness to ~60% duty cycle (153 / 255)
  analogWrite(9, 153);
  delay(500); // Hold this brightness for 500 milliseconds

  // Step 4: Set brightness to ~80% duty cycle (204 / 255)
  analogWrite(9, 204);
  delay(500); // Hold this brightness for 500 milliseconds

  // Step 5: Set brightness to 100% duty cycle (Full brightness)
  analogWrite(9, 255);
  delay(500); // Hold full brightness for 500 milliseconds

  // Step 6: Set brightness to 0% duty cycle (LED completely OFF)
  analogWrite(9, 0);
  delay(500); // Keep the LED off for 500 milliseconds before restarting
  the loop
}
```



Exercise 8: Digital Input-Output Link (Button to LED Control)

Design and program an Arduino system where a digital input device (such as a pushbutton switch) directly mirrors its state onto a digital output component (such as an LED) in real-time.

```
void setup()
{
    // Configure digital pin 4 as an output to drive an LED
    pinMode(4, OUTPUT);

    // Configure digital pin 5 as an input to monitor a switch or button
    pinMode(5, INPUT);
}

void loop()
{
    // 1. Read the current logic state of digital pin 5 (Returns HIGH or LOW)
    int read = digitalRead(5);

    // 2. Transmit that exact state directly to the output on digital pin 4
    // If pin 5 is HIGH, pin 4 becomes HIGH (LED turns ON)
    // If pin 5 is LOW, pin 4 becomes LOW (LED turns OFF)
    digitalWrite(4, read);
}
```

